



CS2022 – Introduction to Database Management Systems



Course Code	Course Title	Hours* (L/P/IL)	Credits	Prereqestic	B.Sc (General)	B.Sc.Hons (Computer Science)
CS2022	Introduction to Database Management Systems	15/30/50	02		✓	✓



Syllabus

- Introduction to Database: Evolution of Database technology, Data models, three level schema architecture, Database languages, Physical file structures and Indexing techniques. Database design: ER model, Relational Model, Normalization - First, Second, Third and Boyce-Codd normal forms. Relational algebra and calculus: Query languages, Relational algebra operations, relational algebra query expressions, tuple calculus and domain calculus query expressions.



Syllabus

- Introduction to SQL: Basic structure, Sub queries, Set operations, Aggregate functions, Derived relations, Views. Advanced Features: Transaction, concurrency control and recovery; Security levels, Integrity constraints. Modern databases: Object Oriented Databases and Relational object Database design, Persistence Objects, Query languages.

References

Recommended Texts:

- Database System Concepts by Abraham Silberschatz, Henry Korth, and S. Sudarshan
- Fundamentals of Database Systems, 5th Edition by [Ramez Elmasri](#), ISBN-13: 978-0321369574.
- Fundamentals of Database Systems, 7th ed., Elmasri & Navathe, Addison-Wesley.

Intended Learning Outcomes (ILOs):

After completing this course, students will be able to

1. Explain the characteristics that distinguish the database approach from the approach of programming with data files. [Familiarity]
2. Describe the most common designs for core database system components including the query optimizer, query executor, storage manager, access methods, and transaction processor. [Familiarity]



Intended Learning Outcomes (ILOs):

3. Describe the components of a database system and give examples of their use. [Familiarity]
4. Explain the database modeling
5. Use a declarative query language to elicit information from a database. [Usage]

Assessment Strategies:

Theory

Summative: Written examination (65%); ILOs Assessed: 1, 2, 3, 4, 5.

Formative: 1. Continuous assessments tests/ quizzes (25%);
2. Assignments (10%)

Assessment Strategies:

Practical

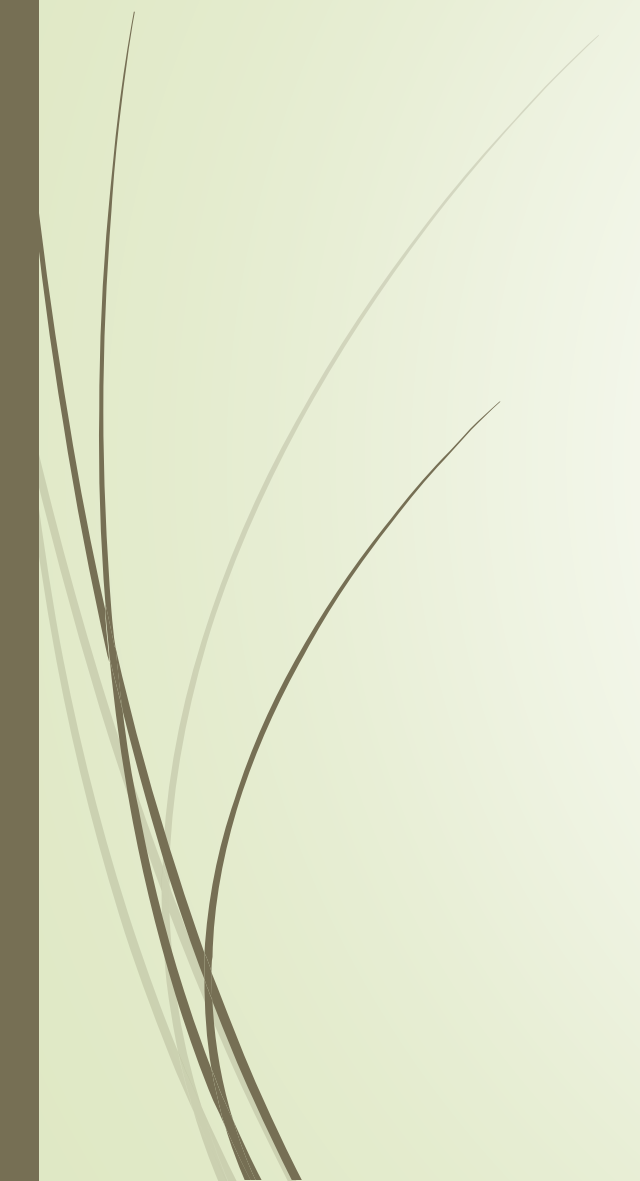
Summative: Lab test (65%); ILOs Assessed: 1, 2, ..., 5.

Formative: Assessments /Home Assignment (35%)

The final marks will be calculated as $(1/2) \times \text{Lecture} + (1/2) \times \text{Practical}$



What is a Database?



What is a Database?

- A **database** is an organized collection of data, generally stored and accessed electronically from a computer system.
- Databases allow data to be stored, managed, retrieved, and updated efficiently.

Evolution of Database Technology

- **File-based systems:** Early data management systems involved using files to store data but lacked efficient management features.
- **Hierarchical & Network Models:** These were among the first data models designed for databases. Hierarchical models had a tree structure, while network models allowed more complex connections.
- **Relational Databases:** Introduced by E.F. Codd in 1970, relational databases store data in tables (relations) and allow for powerful querying.
- **Object-oriented Databases:** Data is stored in the form of objects (as in object-oriented programming).
- **NoSQL Databases:** Modern databases that are designed for large-scale data and unstructured formats, often used in big data and real-time web applications.

Types of Databases:

1. **Relational Databases:** Use tables to represent data and relationships between those tables. Examples include MySQL, PostgreSQL, and Oracle Database.
2. **NoSQL Databases:** Designed for unstructured or semi-structured data, offering flexibility and scalability. Examples include MongoDB, Cassandra, and Redis.
3. **Object-oriented Databases:** Store data in objects, similar to object-oriented programming. Examples include ObjectDB and db4o.

Types of Databases:

4. **Hierarchical Databases:** Organize data in a tree-like structure. Examples include IBM's Information Management System (IMS).
5. **Network Databases:** Use graph structures to represent data and relationships. Examples include Integrated Data Store (IDS) and TurboIMAGE.



Functions of a Database:

1. **Data Storage:** Efficiently store large volumes of data.
2. **Data Retrieval:** Quickly and accurately retrieve specific pieces of data.
3. **Data Manipulation:** Insert, update, and delete data as needed.
4. **Data Security:** Ensure that only authorized users can access or modify data.
5. **Backup and Recovery:** Protect data against loss and provide mechanisms to recover data in case of failures.

Examples of Database Usage

1. **Business Applications:** Customer Relationship Management (CRM), Enterprise Resource Planning (ERP) systems.
2. **Web Applications:** Content Management Systems (CMS), e-commerce platforms.
3. **Scientific Research:** Storing and analyzing large datasets from experiments.
4. **Finance:** Managing transactions, accounts, and financial records.
5. **Healthcare:** Storing patient records, medical histories, and research data.



Disadvantages of file processing system

- ➡ 1. Data redundancy and Inconsistency
- ➡ 2. Difficulty in accessing data
- ➡ 3. Data isolation
- ➡ 4. Integrity problems
- ➡ 5. Concurrent access anomalies
- ➡ 6. Security problems

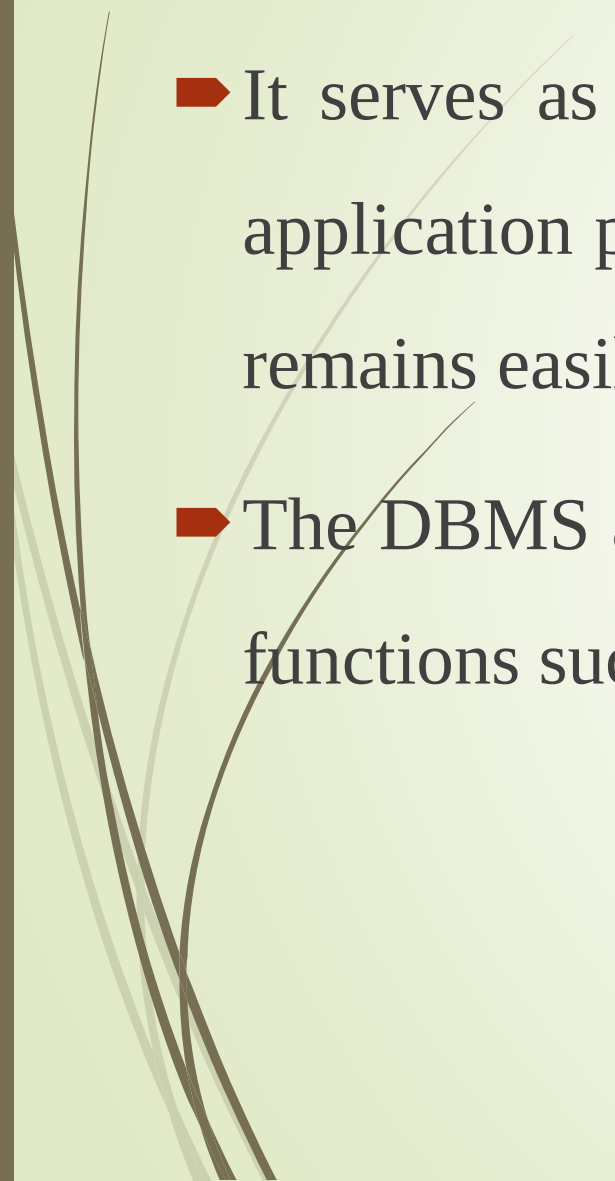


Database Management System (DBMS)

- A Database Management System (DBMS) is software designed to define, create, maintain, and control access to databases.
- 



Database Management System (DBMS)

- It serves as an intermediary between the database and the users or application programs, ensuring that data is consistently organized and remains easily accessible.
 - The DBMS allows users to interact with the database through various functions such as data querying, updating, and administration.
- 

Types of DBMS

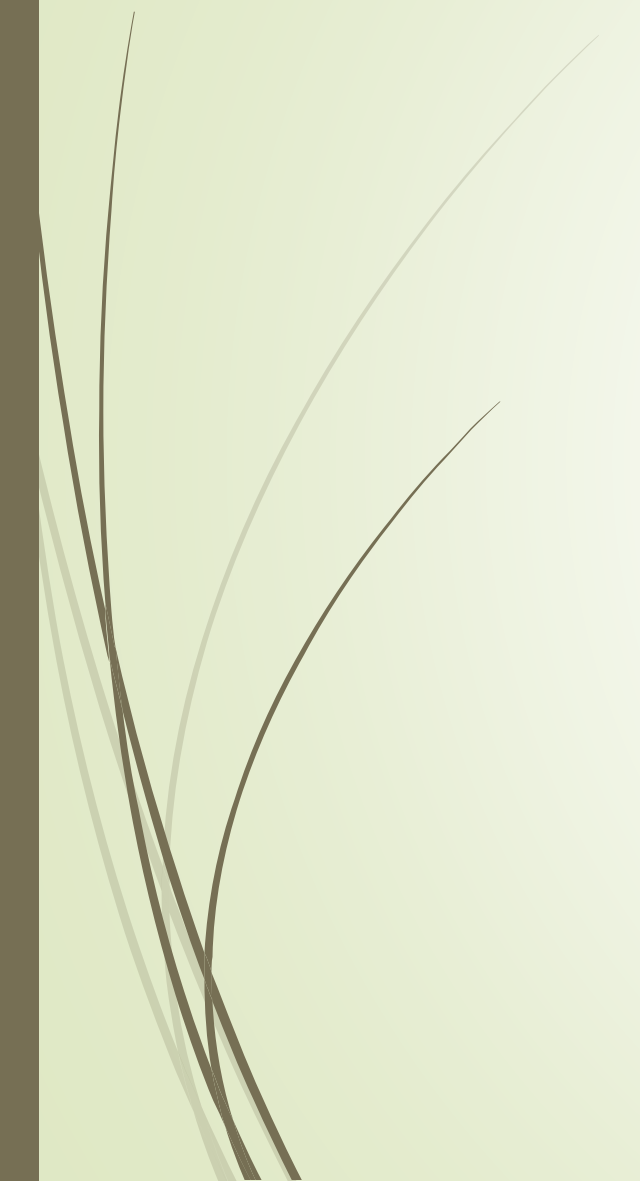
1. **Hierarchical DBMS:** Organizes data in a tree-like structure with parent-child relationships.
2. **Network DBMS:** Uses a graph structure to represent data and relationships, allowing many-to-many relationships.
3. **Relational DBMS (RDBMS):** Represents data in tables (relations) and uses SQL for data manipulation. Examples include MySQL, PostgreSQL, Oracle, and SQL Server.
4. **Object-oriented DBMS:** Stores data in objects, similar to object-oriented programming. Examples include ObjectDB and db4o.

Types of DBMS

5. **NoSQL DBMS:** Designed for unstructured or semi-structured data, providing flexibility and scalability. Examples include MongoDB, Cassandra, Redis, and Couchbase.
6. **NewSQL DBMS:** Combines the benefits of traditional RDBMS with the scalability of NoSQL. Examples include CockroachDB and Google Spanner.



Advantages of DBMS





Advantages of DBMS

- ➡ 1. Reducing Data Redundancy
- ➡ 2. Sharing of Data
- ➡ 3. Data Integrity
- ➡ 4. Data Security
- ➡ 5. Privacy

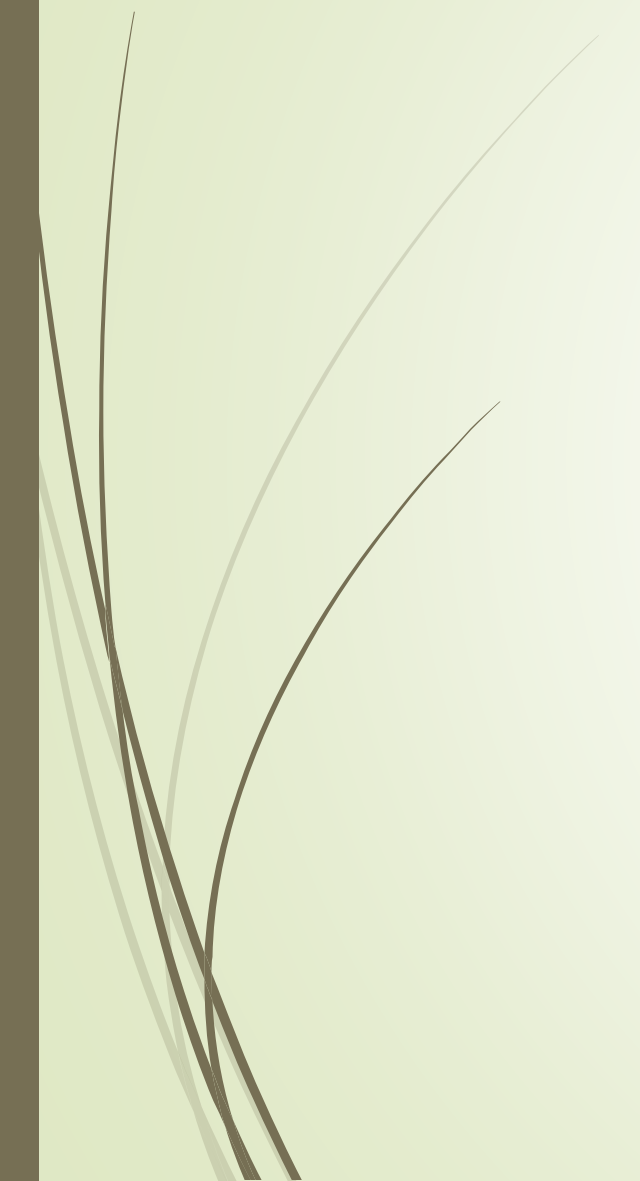


Advantages of DBMS

- 6. Backup and Recovery
 - 7. Data Consistency
- 

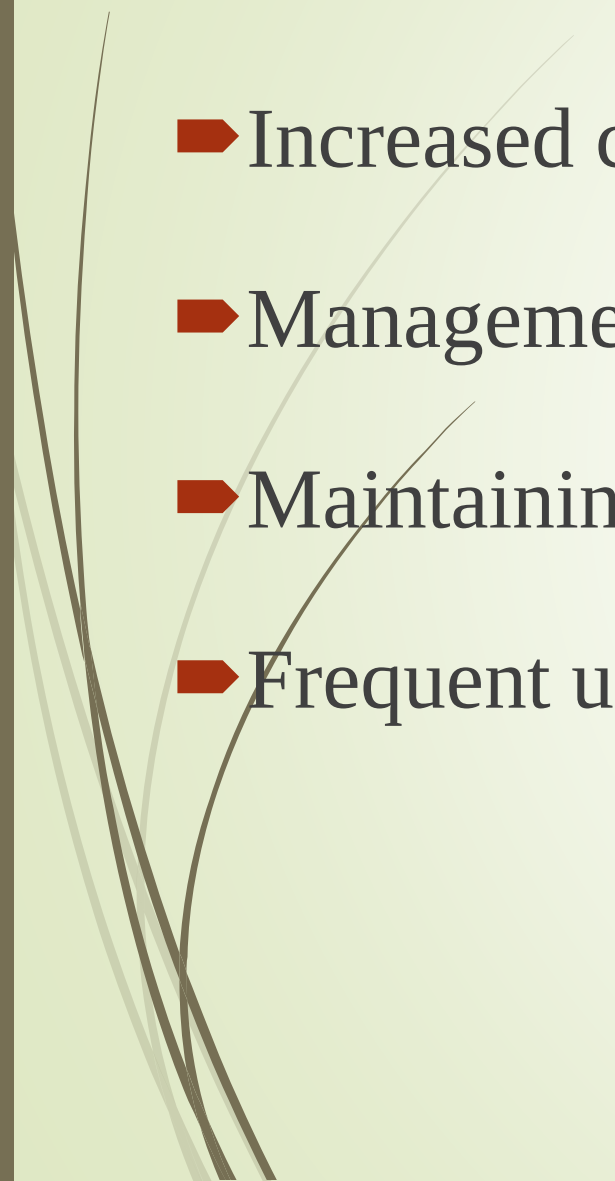


Disadvantages of DBMS





Disadvantages of DBMS

- Increased costs
 - Management complexity
 - Maintaining currency
 - Frequent upgrade/replacement cycles
- 

When Not to Use a DBMS

- While Database Management Systems (DBMS) are highly beneficial for data organization and management, there are certain scenarios where using a DBMS might not be the best option.

When Not to Use a DBMS

1. Small or Simple Data Requirements

- **Why:** If the data is minimal and straightforward, managing it with simple tools (e.g., flat files, spreadsheets) may be more efficient.
- **Example:** A small business tracking employee hours might find a spreadsheet sufficient for their needs without the complexity of a DBMS.

When Not to Use a DBMS

2. High Initial Investment Costs

- **Why:** Setting up and maintaining a DBMS can be expensive, including costs for hardware, software, and specialized personnel.
- **Example:** A startup or small organization with limited resources may avoid DBMS due to these costs.

When Not to Use a DBMS

3. Overhead and Complexity

- **Why:** A DBMS introduces overhead for tasks like backup, performance tuning, security management, and ensuring data integrity.
- **Example:** For applications where simplicity is key and data management requirements are minimal, adding this overhead can be unnecessary and burdensome.

When Not to Use a DBMS

4. Real-Time Applications with Extreme Performance Needs

- **Why:** DBMSs, while efficient, may introduce latency due to their layers of abstraction and transaction management. Real-time systems may need faster, low-latency data access.
- **Example:** High-frequency trading systems or real-time embedded systems may require custom-built, optimized data management solutions rather than a full DBMS.

When Not to Use a DBMS

5. Limited Multi-User Access Requirements

- **Why:** If there is no need for multi-user access, the multi-user capabilities of a DBMS might be unnecessary.
- **Example:** A personal project with a single user can store and access data via local files without the need for a DBMS's multi-user handling.

When Not to Use a DBMS

6. No Complex Querying Requirements

- **Why:** DBMSs are valuable when you need complex querying and reporting functionalities. If you don't need to run sophisticated queries, a DBMS might be overkill.
- **Example:** Logging system data (e.g., application logs) where no complex queries are necessary may not require a DBMS and can be stored in log files.

When Not to Use a DBMS

7. Highly Static Data

- **Why:** If the data rarely changes and doesn't require updates or frequent retrieval, a DBMS might add unnecessary complexity.
- **Example:** Archival systems where data is rarely accessed or modified might not need the dynamic features of a DBMS.

When Not to Use a DBMS

8. Non-critical Data Storage

- **Why:** If the data is not critical and the risks of data loss or corruption are minimal, the robustness of a DBMS may not be needed.
- **Example:** Temporary or cache data storage, where data loss wouldn't affect the business operations.

When Not to Use a DBMS

9. Lack of Skilled Personnel

- **Why:** A DBMS requires technical knowledge for configuration, optimization, and management. If an organization lacks skilled DBMS administrators, implementing and maintaining it can be challenging.
- **Example:** Small organizations without access to trained database administrators may struggle with the complexities of a DBMS.

When Not to Use a DBMS

10. Data Integrity and Security Not a Concern

- **Why:** If data integrity and security are not a primary concern, the additional features of a DBMS for transaction management, integrity constraints, and access control may be unnecessary.
- **Example:** Some local development environments or non-critical applications may prefer simple file systems over a DBMS if data security is not a concern.

Three level schema architecture

- The three-level schema architecture is a framework that defines
 - how data is structured,
 - stored, and
 - accessed in a database system.
- It helps in separating the user's view of the database from the way it is physically stored, promoting data independence.

Three level schema architecture

- Introduced as part of the ANSI/SPARC (American National Standards Institute/Standards Planning and Requirements Committee) database model in the 1970s.
- Provides a way to abstract the complexities of how data is physically stored from how users view and interact with the data.

Three level schema architecture

The architecture consists of three levels:

1. External level (user view)
2. Conceptual level (community view)
3. Internal level (physical storage)

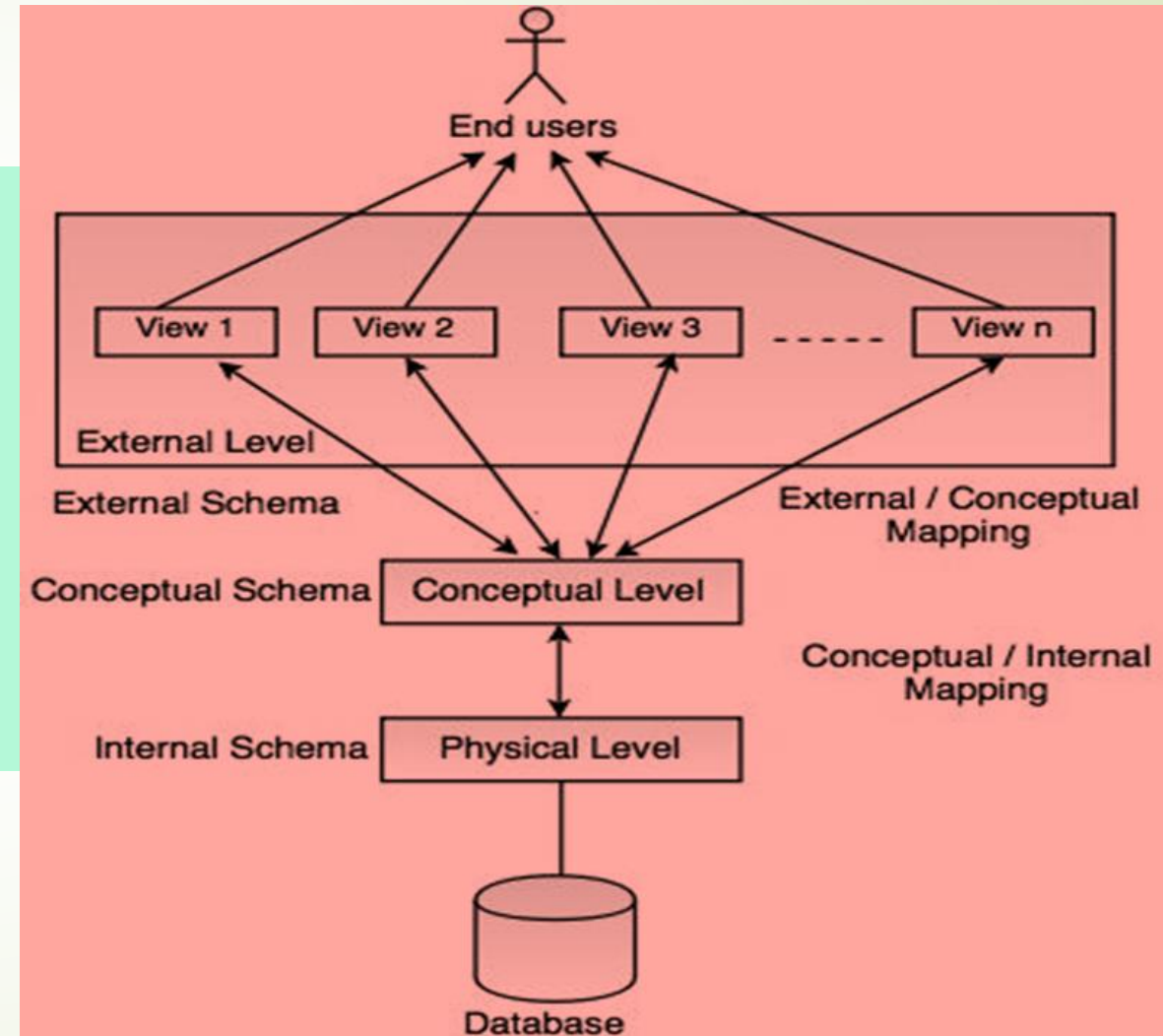


Fig. Three Level Architecture of DBMS

Three level schema architecture

- External Level (User View)
 - The external level defines how individual users or user groups view the data in the database.
 - It consists of multiple external schemas, each corresponding to a particular user's or application's view.

Example: In a university database, a student may only see their personal details and grades, while an admin may have access to all students' records and financial data.

Three level schema architecture

- Conceptual Level (Logical View)
 - The conceptual level provides a logical view of the entire database as seen by the community of users.
 - It defines what data is stored in the database and the relationships among the data.

Example: In the university database, the conceptual schema would describe entities like **students**, **courses**, **professors**, and the relationships like students **enrolled in** courses or professors **teaching** courses.

Three level schema architecture

- Internal Level (Physical Storage)
 - The internal level describes how the data is actually stored on the physical medium.
 - It deals with the physical storage of data on disks, file structures, indexing, and access methods.

Example: In the university database, the internal schema may define how student and course data is stored on a hard disk, how indexes are created for fast retrieval, and how data blocks are organized for optimized performance.

Classification of DBMS Users

- In a Database Management System (DBMS), different types of users interact with the system in various ways depending on their roles and the tasks they need to perform.

Classification of DBMS Users

- ➡ Database Administrators (DBAs)
- ➡ Database Designers
- ➡ End Users
- ➡ Application Programmers/Developers
- ➡ System Analysts



Classification of DBMS Users

1. Database Administrators (DBAs)

- Role: Responsible for overall control of the database.
- Functions:
 - Creating and managing the database structure.
 - Authorizing user access to the database.
 - Ensuring database security.
 - Monitoring system performance.
 - Handling backups and recovery.
 - Installing, configuring, and upgrading the DBMS software.

Classification of DBMS Users

2. Database Designers

- Role: Define the structure of the database.
- Functions:
 - Determining the data to be stored and how it will be organized.
 - Designing the schema at various levels (conceptual, physical, and external).
 - Establishing relationships between different data elements.

Classification of DBMS Users

3. End Users

- **Role:** Access the database to perform various operations.
- **Classification of End Users:**
 - **Casual Users:**
 - Interact with the database occasionally.
 - Use query languages like SQL or graphical user interfaces.
 - Do not require in-depth knowledge of the DBMS.

Classification of DBMS Users

- **Naive or Parametric Users:**
 - Use pre-defined applications and forms to perform simple tasks.
 - Examples: Bank tellers, reservation clerks.
- **Sophisticated Users:**
 - Have advanced knowledge of DBMS and interact through complex queries or development of database applications.
 - Examples: Scientists, business analysts.

Classification of DBMS Users

- **Standalone Users:**
 - Use ready-made database applications such as personal database systems.
 - Example: A small business owner using a personal finance system.

Classification of DBMS Users

4. Application Programmers/Developers

- **Role:** Develop and maintain applications that use the database.
- **Functions:**
 - Writing code to interact with the database.
 - Implementing business logic to process the data.
 - Developing front-end applications that interact with users.
- **Languages Used:** SQL, Python, Java, etc.

Classification of DBMS Users

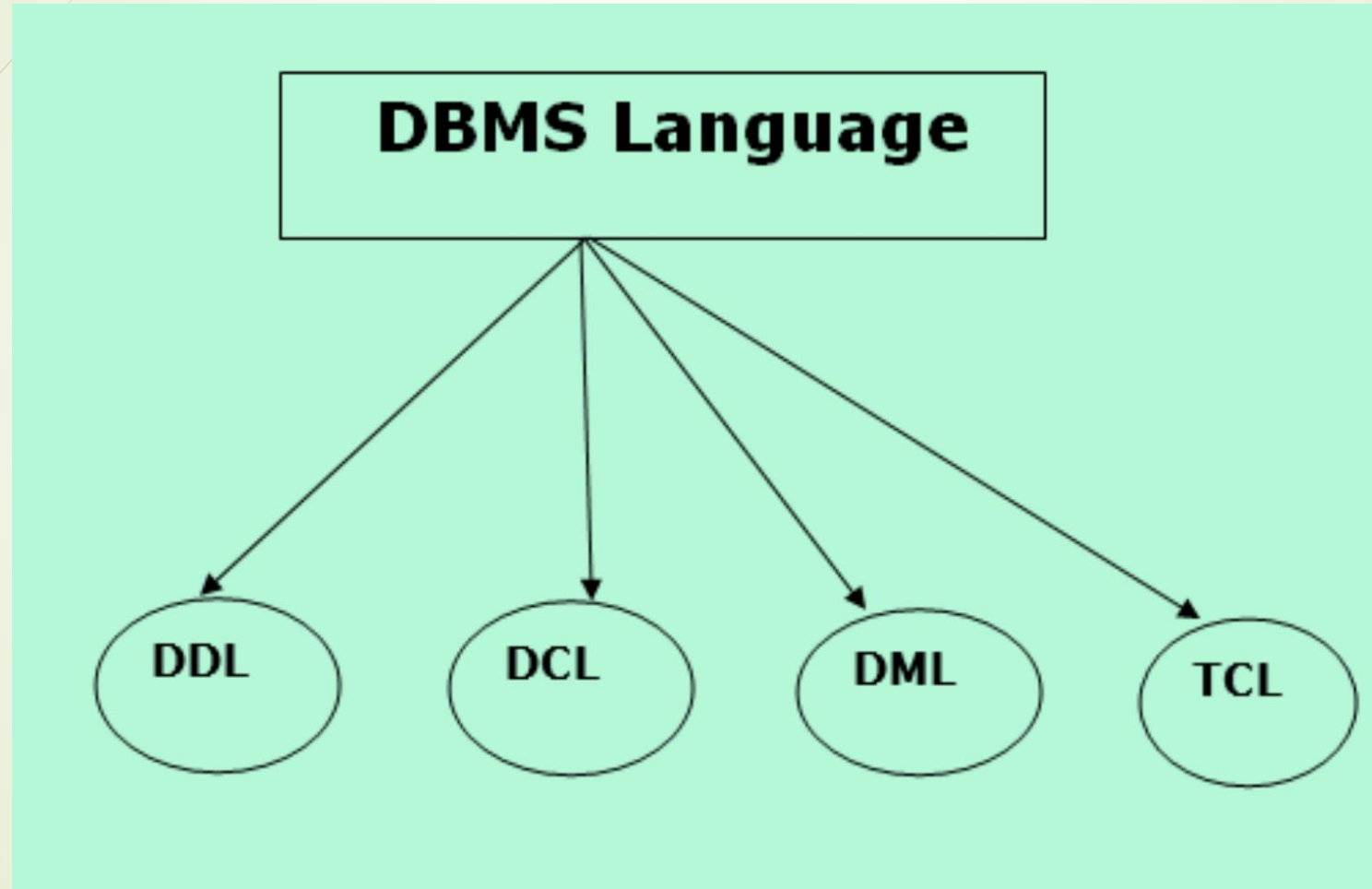
5. System Analysts

- **Role:** Understand the requirements of the users and design database solutions.
- **Functions:**
 - Analyzing the organization's data requirements.
 - Designing systems and solutions that meet these requirements.
 - Working closely with DBAs and developers to ensure successful implementation.

Database Languages

- Database languages are used to interact with and manage a Database Management System (DBMS).
- They allow users to create, update, manipulate, and retrieve data efficiently.
- DBMS languages are classified into different categories based on their functions.

Database Languages



Database Languages

1. Data Definition Language (DDL)

- **Purpose:** To define the structure and schema of a database.
- **Functions:**
 - Creating, altering, and deleting database objects such as tables, views, indexes, etc.
 - Defining the relationships between tables.
- **Common DDL Commands:**
 - **CREATE** : Creates new database objects (tables, indexes, etc.).
 - **ALTER** : Modifies existing database structures (adding or removing columns, etc.).
 - **DROP** : Deletes database objects.
 - **TRUNCATE** : Removes all data from a table but retains its structure.

Database Languages

2. Data Manipulation Language (DML)

- Purpose: To manipulate the data within the database.
- Functions:
 - Inserting, updating, deleting, and retrieving data from tables.
- Common DML Commands:
 - `INSERT` : Adds new data into tables.
 - `UPDATE` : Modifies existing data in tables.
 - `DELETE` : Removes data from tables.
 - `SELECT` : Retrieves data from tables.

Database Languages

3. Data Control Language (DCL)

- Purpose: To control access to data in the database.
- Functions:
 - Granting and revoking user access rights and privileges.
- Common DCL Commands:
 - **GRANT** : Gives specific privileges to users or roles.
 - **REVOKE** : Removes previously granted privileges from users or roles.

Database Languages

4. Transaction Control Language (TCL)

- **Purpose:** To manage transactions in the database.
- **Functions:**
 - Ensuring the integrity of data during transactions by controlling commit and rollback operations.
- **Common TCL Commands:**
 - **COMMIT** : Saves all changes made during the current transaction.
 - **ROLLBACK** : Reverts the database to its previous state by undoing changes made in the current transaction.
 - **SAVEPOINT** : Sets a point in a transaction to which a rollback can be made.

Database Languages

Summary of Database Language Categories:

Category	Purpose	Common Commands
DDL	Defines database structure and schema	CREATE , ALTER , DROP , TRUNCATE
DML	Manipulates and modifies data	INSERT , UPDATE , DELETE , SELECT
DCL	Controls access to data	GRANT , REVOKE
TCL	Manages transactions	COMMIT , ROLLBACK , SAVEPOINT

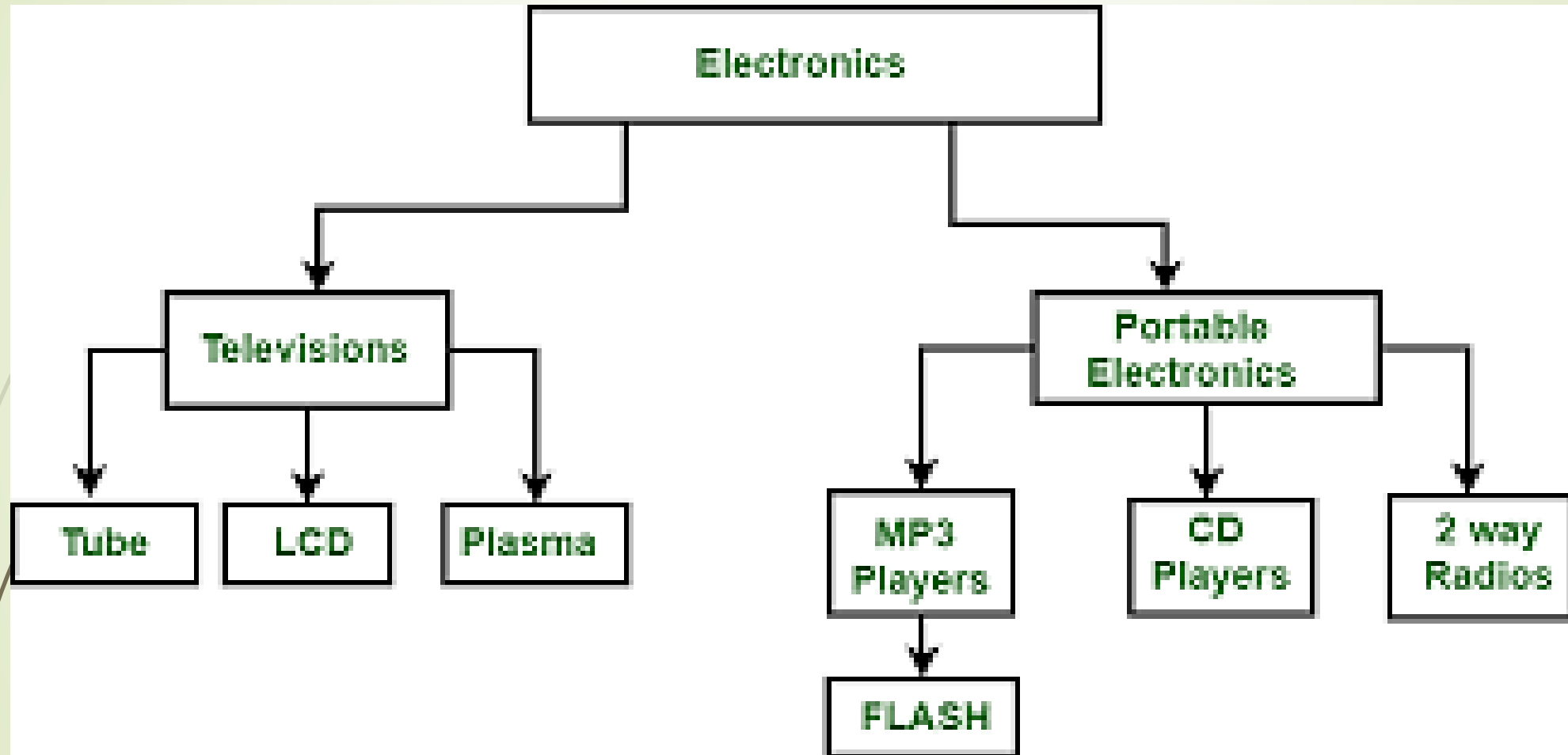
Data Models in DBMS

- A **Data Model** defines **how data is logically structured, organized, and manipulated** within a database management system (DBMS).
- It provides the abstraction needed to represent the complex real-world data in a simplified form.
- Data models determine **how data is stored, processed, and accessed.**

Data Models in DBMS

- Hierarchical Model
- Network Model
- Entity-Relationship Model
- Relational Model
- Object-Oriented Data Model
- Object-Relational Data Model
- Flat Data Model
- Semi-Structured Data Model
- Associative Data Model
- Context Data Model

Hierarchical Data Model



Hierarchical Data Model

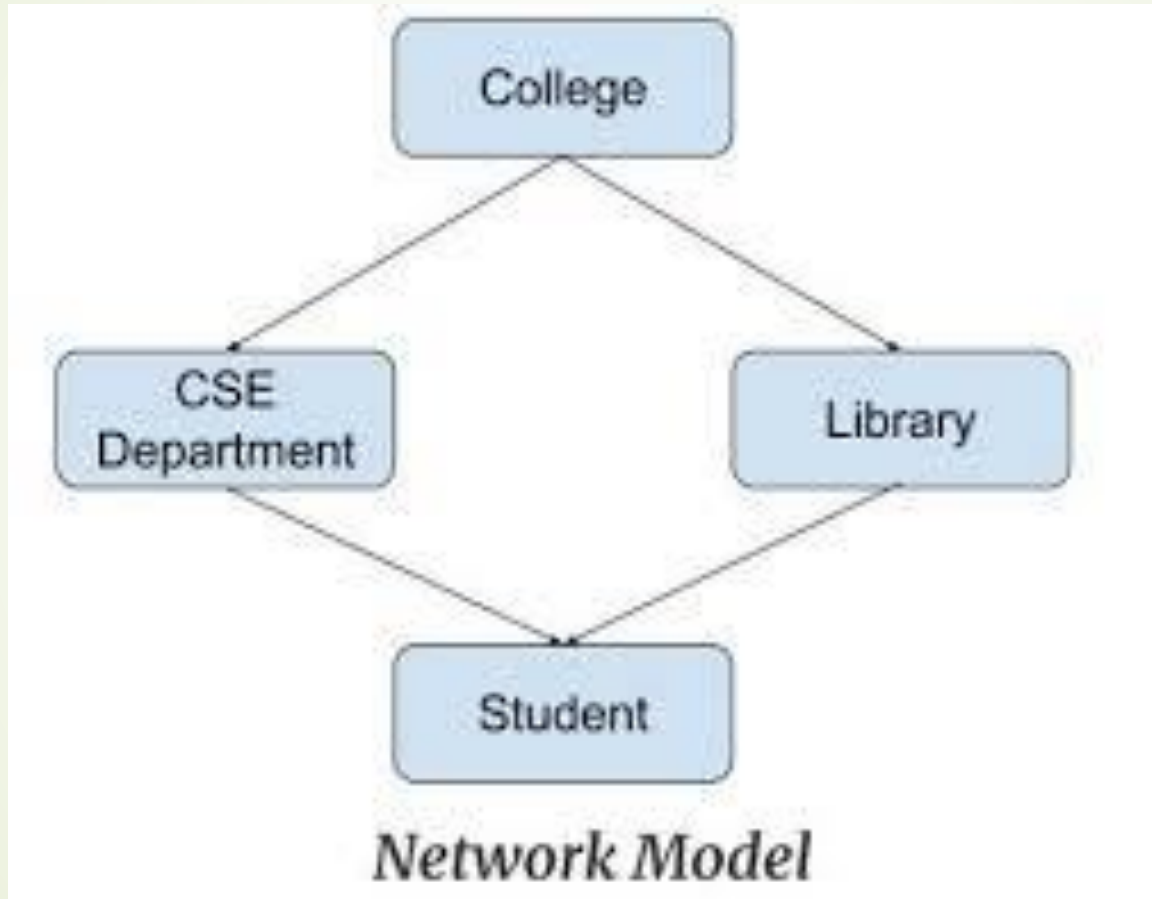
- Structure: Tree-like structure.
- Description:
 - Data is organized in a hierarchical (parent-child) relationship.
 - Each parent can have multiple children, but each child has only one parent.
- Example:
 - An organization's structure where a manager (parent) supervises employees (children).



Hierarchical Data Model

- **Advantages:**
 - Simple to understand and implement.
 - Efficient for one-to-many relationships.
- **Disadvantages:**
 - Difficult to represent complex many-to-many relationships.
 - Limited flexibility in accessing data.

Network Data Model



Network Data Model

- **Structure:** Graph-like structure.
- **Description:**
 - A more flexible model compared to the hierarchical model.
 - Data is organized as records connected by links, allowing for many-to-many relationships.
 - Each record (node) can have multiple parents and multiple children.
- **Example:**
 - A university system where students are enrolled in multiple courses, and courses have multiple students.

Network Data Model

- **Advantages:**
 - More flexible than the hierarchical model.
 - Efficient handling of complex relationships.
- **Disadvantages:**
 - Complex to design and maintain.
 - Querying can be difficult.

Relational Data Model

Relational Model

Column 1	Column 2	Column 3	Column 4
↑	↑	↑	↑
Roll_No	Name	Age	GPA
1	Arya	21	4
2	Bran	19	3
3	John	24	4.3
4	Max	24	1

→ Row 1
→ Row 2
→ Row 3
→ Row 4

Relational Data Model

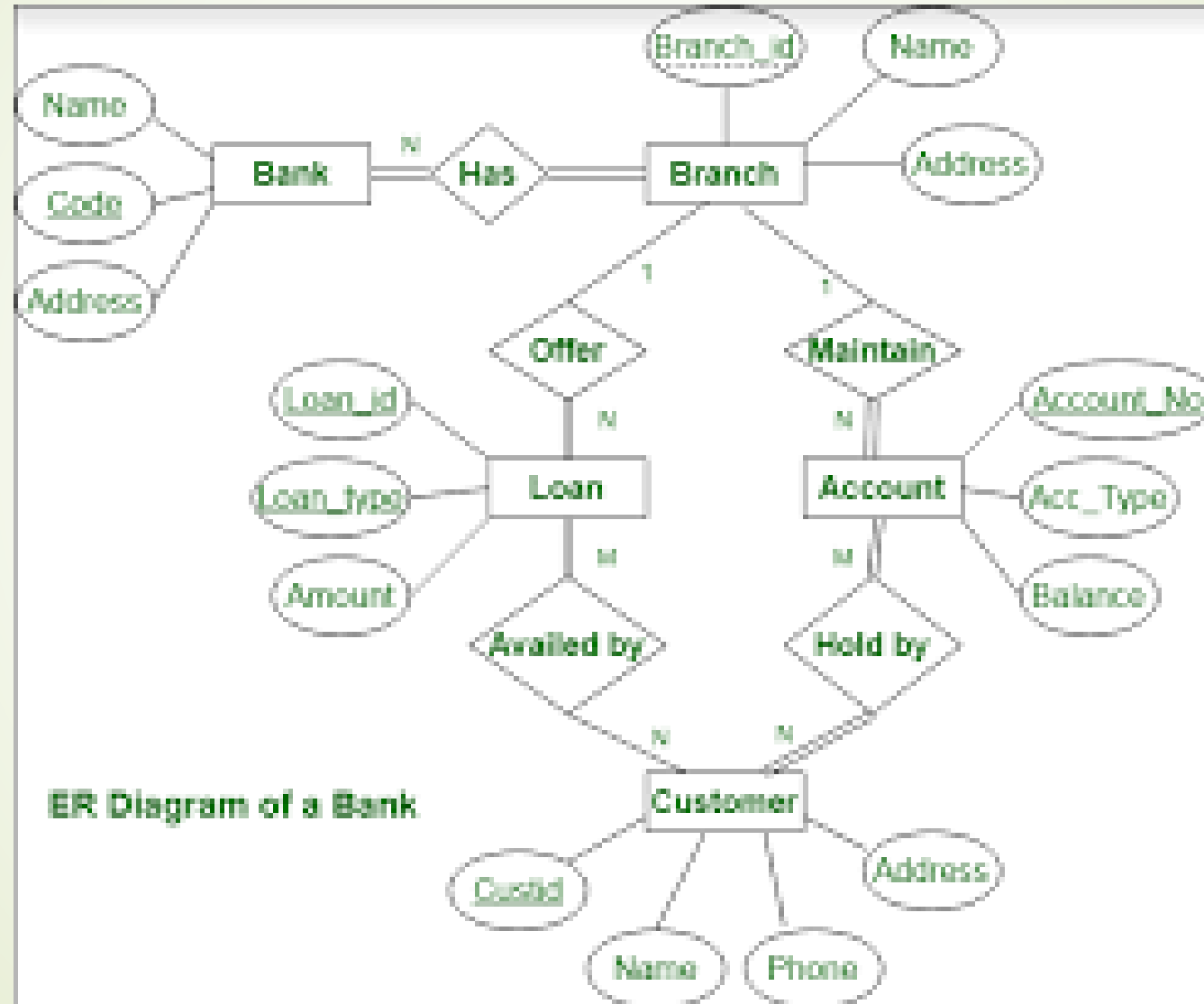
- **Structure:** Table-based structure.
- **Description:**
 - Data is stored in **tables (relations)**, with rows representing records and columns representing attributes.
 - Tables can be related to each other using keys (primary key and foreign key).
- **Example:**
 - A customer database where each customer is a record in the "Customers" table, and their orders are stored in an "Orders" table.



Relational Data Model

- **Advantages:**
 - Simple and widely used.
 - Supports complex queries through SQL.
 - Allows flexibility in accessing data.
- **Disadvantages:**
 - Can become inefficient with very large datasets.
 - Limited support for complex data types (before extensions like NoSQL).

Entity-Relationship (ER) Model



Entity-Relationship (ER) Model

- **Structure:** Diagrammatic (ER diagrams).
- **Description:**
 - The ER model represents real-world entities (like people, objects) and relationships between them.
 - Entities have attributes, and relationships can be one-to-one, one-to-many, or many-to-many.
- **Example:**
 - A database of students and courses where "Students" and "Courses" are entities, and the relationship "Enrolled" connects them.

Entity-Relationship (ER) Model

- **Advantages:**
 - Intuitive and easy to understand.
 - Useful in the conceptual design phase of databases.
- **Disadvantages:**
 - Needs to be translated into other models (like relational) for implementation.

Questions

- Discuss the advantages of using a DBMS over traditional file-based systems using three valid points.

Questions

- Define the term *Data Model* and explain its role in representing data in a database

Questions

- ➡ Write three advantages of DBMS.

Questions

- Data definition language and data manipulation language are two types of languages used in database management systems. Briefly describe them.



Entity

In **Database Management Systems (DBMS)**, an **entity** is an object, concept, or thing in the real world that can be uniquely identified and stored as data in a database. Each entity represents something that has a meaningful existence in the context of the database.



Entity

1. In a University Database:

- **Student:** Attributes could include `Student_ID`, `Name`, `Age`, `Major`, `Enrollment_Date`.
- **Course:** Attributes could include `Course_ID`, `Course_Name`, `Credits`, `Instructor`.
- **Instructor:** Attributes could include `Instructor_ID`, `Name`, `Department`, `Email`.

2. In a Retail Store Database:

- **Customer:** Attributes could include `Customer_ID`, `Name`, `Address`, `Phone_Number`, `Email`.
- **Product:** Attributes could include `Product_ID`, `Product_Name`, `Price`, `Category`, `Stock`.
- **Order:** Attributes could include `Order_ID`, `Order_Date`, `Customer_ID`, `Total_Amount`.

Entity

3. In a Library Management System:

- **Book:** Attributes could include `Book_ID`, `Title`, `Author`, `Publisher`, `Genre`.
- **Member:** Attributes could include `Member_ID`, `Name`, `Address`, `Phone_Number`, `Membership_Type`.
- **Loan:** Attributes could include `Loan_ID`, `Member_ID`, `Book_ID`, `Issue_Date`, `Return_Date`.

4. In an Employee Management System:

- **Employee:** Attributes could include `Employee_ID`, `Name`, `Position`, `Department`, `Salary`.
- **Department:** Attributes could include `Department_ID`, `Department_Name`, `Manager_ID`.
- **Project:** Attributes could include `Project_ID`, `Project_Name`, `Start_Date`, `End_Date`, `Budget`.

Attributes

In the context of databases and data modeling, **attributes** are the properties or characteristics that describe an entity. Each attribute provides specific details about the entity and is typically represented as a field or column in a database.

Key Aspects of Attributes

1. **Defines Characteristics:** Attributes provide information about an entity. For example, in a "Student" entity, attributes like `Name` , `Age` , and `Student_ID` describe characteristics of a student.

Key Aspects of Attributes

2. Types of Attributes:

- **Simple Attributes:** Attributes that cannot be broken down further, like `Age` or `Gender`.
- **Composite Attributes:** Attributes that can be divided into smaller sub-attributes, like `Address` (which might include `Street`, `City`, `State`, and `Zip Code`).
- **Single-Valued Attributes:** Attributes that have only one value, like `Employee_ID` for each employee.
- **Multi-Valued Attributes:** Attributes that can have multiple values, like `Phone_Number` if a person has more than one phone.
- **Derived Attributes:** Attributes calculated from other attributes, like `Age` derived from `Date_of_Birth`.

Key Aspects of Attributes

3. **Primary Key Attribute:** A unique attribute (or a set of attributes) that uniquely identifies each entity in an entity set. For example, `Student_ID` could be the primary key for a "Student" entity.

Key Aspects of Attributes

Examples of Attributes by Entity

- Student: Student_ID, Name, Age, Major
- Employee: Employee_ID, Name, Position, Salary
- Product: Product_ID, Product_Name, Price, Category

Keys

1. Super Key

A **super key** is any combination of columns that can uniquely identify a row in a table. A super key can have extra attributes beyond those needed for uniqueness.

Example: For a table `Employee` with attributes `Employee_ID`, `Name`, and `Department`, the following are examples of super keys:

- `{Employee_ID}`
- `{Employee_ID, Name}`
- `{Employee_ID, Department}`

Keys

2. Candidate Key

A **candidate key** is a minimal super key, meaning it is the smallest possible combination of attributes that uniquely identifies a row. A table can have multiple candidate keys, but each one must be minimal and unique.

Example: In the same `Employee` table with attributes `Employee_ID`, `Name`, and `Department`:

- `{Employee_ID}` is a candidate key (since it's minimal and unique).
- If `Employee_ID` is unique and sufficient, other combinations (like `{Employee_ID, Name}`) would not be candidate keys, as they include unnecessary attributes.

Keys

3. Composite Key

A **composite key** is a candidate key made up of two or more attributes that together uniquely identify a row. It is used when a single attribute alone is insufficient to ensure uniqueness.

Example: In a `Course_Enrollment` table with attributes `Student_ID`, `Course_ID`, and `Enrollment_Date`, neither `Student_ID` nor `Course_ID` alone can uniquely identify a row because a student may enroll in multiple courses. However, the combination `{Student_ID, Course_ID}` is unique and serves as a composite key for this table.

Keys

4. Foreign Key

A **foreign key** is an attribute or set of attributes in one table that references the primary key of another table. This key establishes a relationship between two tables and enforces referential integrity.

Example: In a `Orders` table with attributes `Order_ID`, `Customer_ID`, and `Order_Date`:

- `Customer_ID` could be a foreign key referencing the `Customer_ID` primary key in a `Customers` table.
- This relationship links orders to the customers who placed them, ensuring that each `Customer_ID` in `Orders` must exist in the `Customers` table.

Primary Keys

A **primary key** is a specific type of candidate key in a database table that uniquely identifies each record (or row) in the table. Each table can have only one primary key, and it is chosen from the set of candidate keys. The primary key must contain unique values and cannot contain `NULL` values, ensuring that every row is distinct and identifiable.

Primary Keys

Characteristics of a Primary Key:

1. **Uniqueness:** No two rows can have the same value for the primary key, which ensures each row can be uniquely identified.
2. **Non-nullability:** The primary key column(s) cannot contain `NULL` values.
3. **Immutability:** Ideally, the values of the primary key should not change, as this can complicate data integrity and relationships.

Example of a Primary Key

Consider a `Student` table with the following columns:

- `Student_ID` (unique for each student)
- `Name`
- `Age`
- `Email`

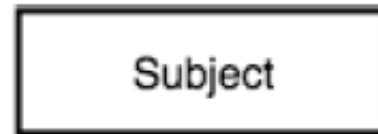
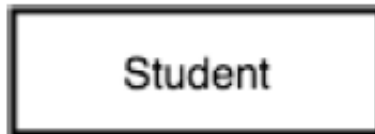
Keys

Key Type	Definition	Example
Super Key	Any combination of columns that uniquely identifies a row.	{Employee_ID} , {Employee_ID, Department}
Candidate Key	Minimal super key (no unnecessary attributes) that uniquely identifies a row.	{Employee_ID}
Composite Key	A key consisting of multiple attributes used to uniquely identify a row.	{Student_ID, Course_ID} in Course_Enrollment
Foreign Key	An attribute(s) in one table that references the primary key of another, establishing a relationship.	Customer_ID in Orders table referencing Customer_ID in Customers table

Entity

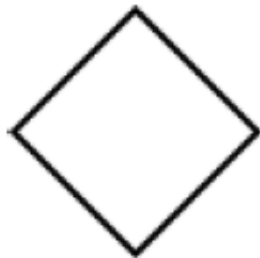
Entity

Simple rectangular box represents an Entity.



Relationships between Entities - Weak and Strong

Rhombus is used to setup relationships between two or more entities.



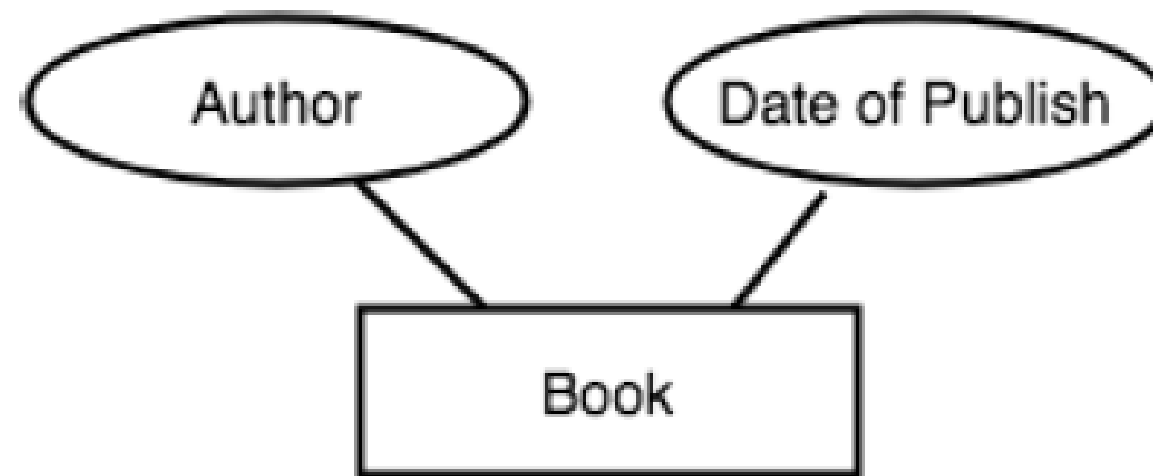
Relationship



Weak
Relationship

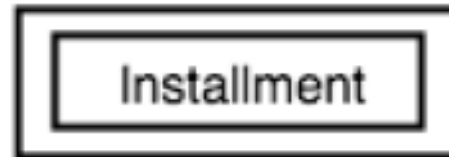
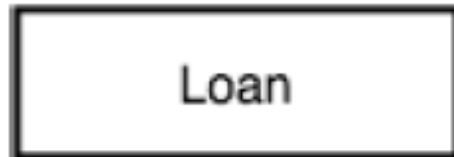
Attributes for any Entity

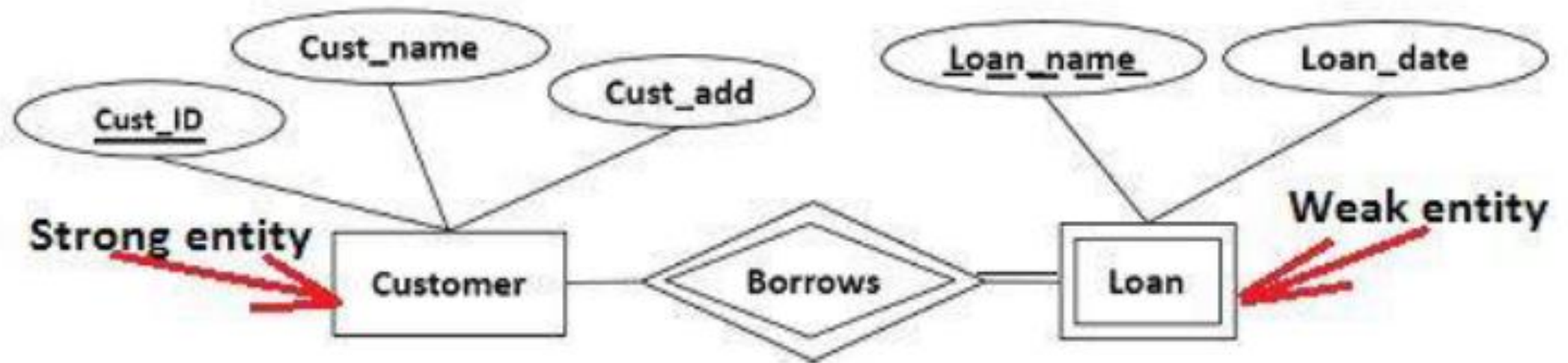
Ellipse is used to represent attributes of any entity. It is connected to the entity.



Weak Entity

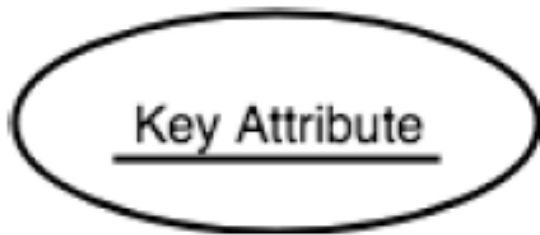
A weak Entity is represented using double rectangular boxes. It is generally connected to another entity.





Key Attribute for any Entity

To represent a Key attribute, the attribute name inside the Ellipse is *underlined*.



Derived Attribute for any Entity

Derived attributes are those which are derived based on other attributes, for example, age can be derived from date of birth. To represent a derived attribute, another dotted ellipse is created inside the main ellipse.



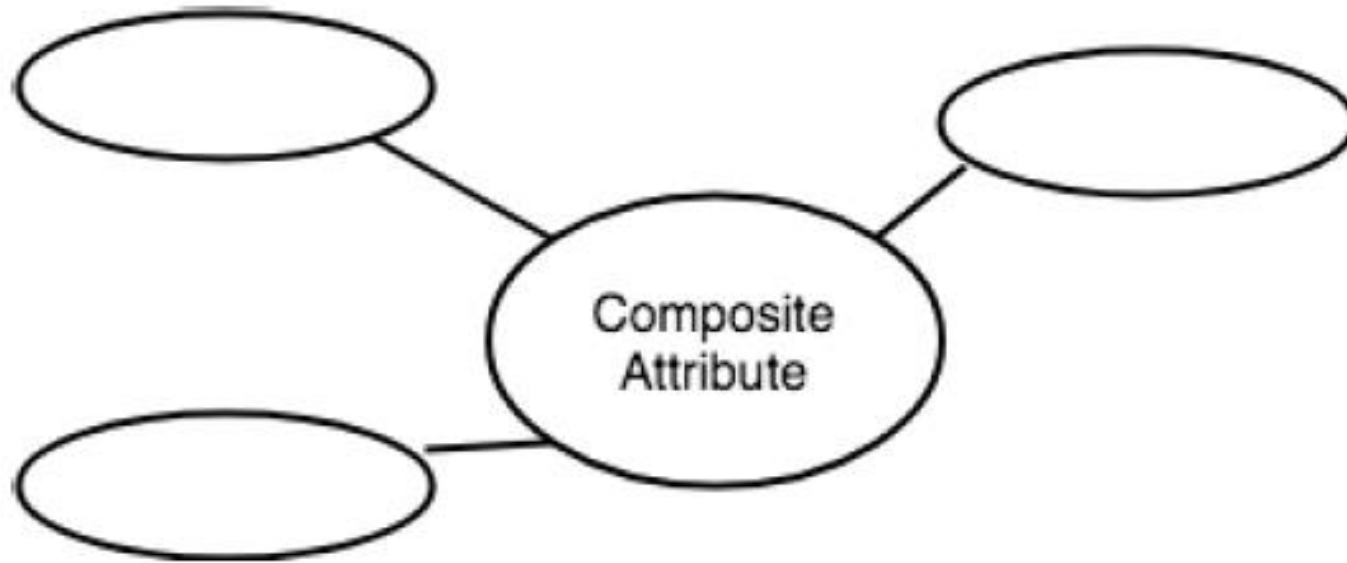
Multivalued Attribute for any Entity

Double Ellipse, one inside another, represents the attribute which can have multiple values.



Composite Attribute for any Entity

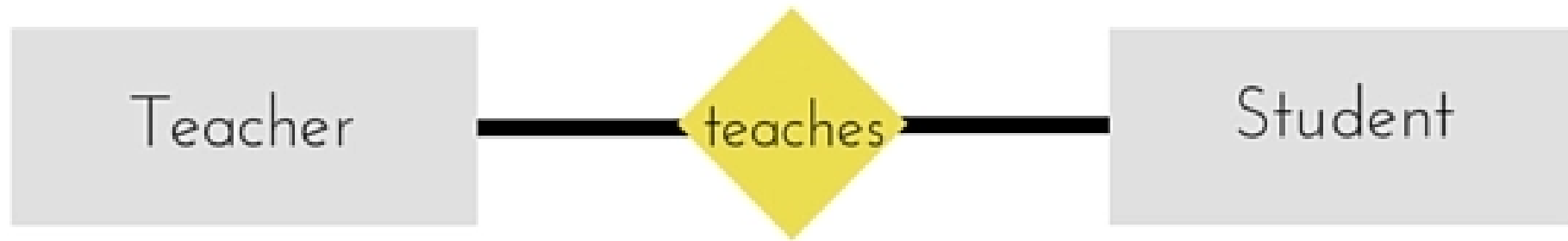
A composite attribute is the attribute, which also has attributes.



ER Diagram: Entity



ER Diagram: Entity



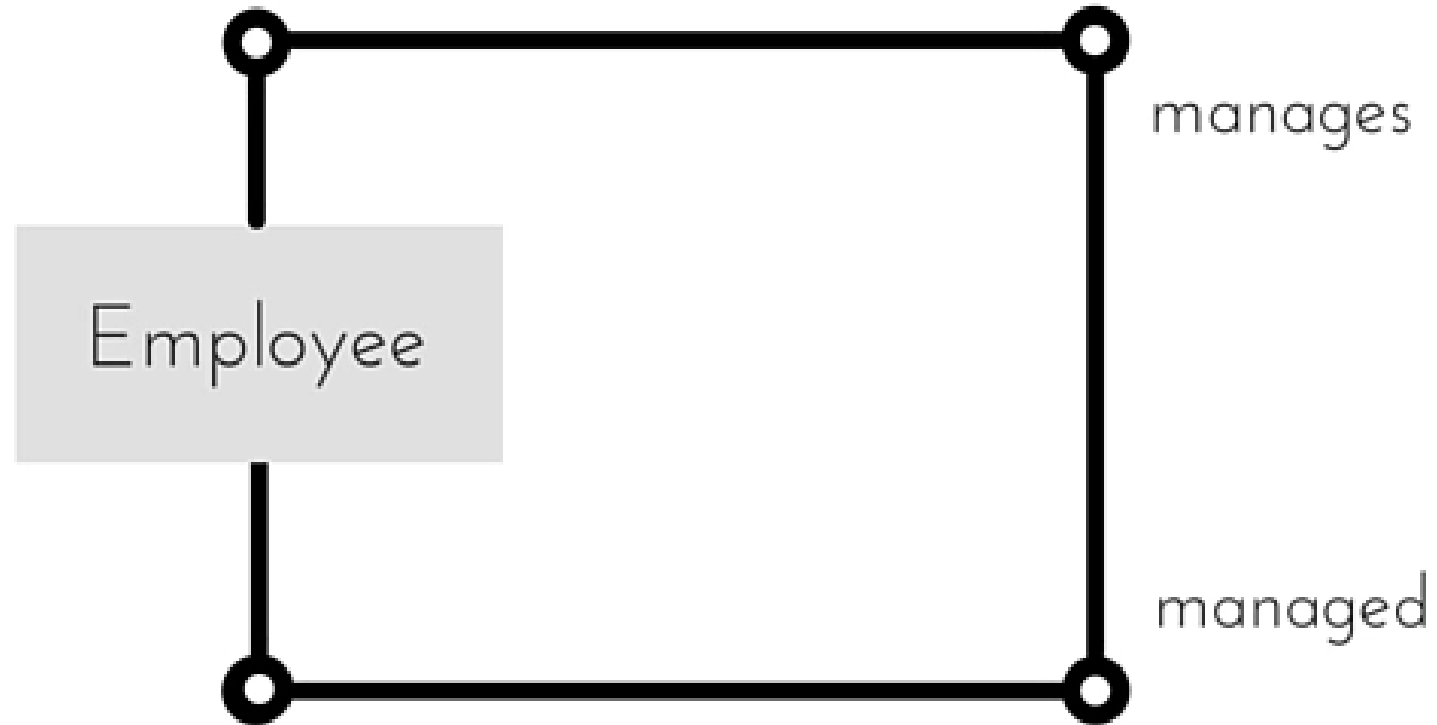
ER Diagram: Entity

- 1. Binary Relationship
- 2. Recursive Relationship
- 3. Ternary Relationship

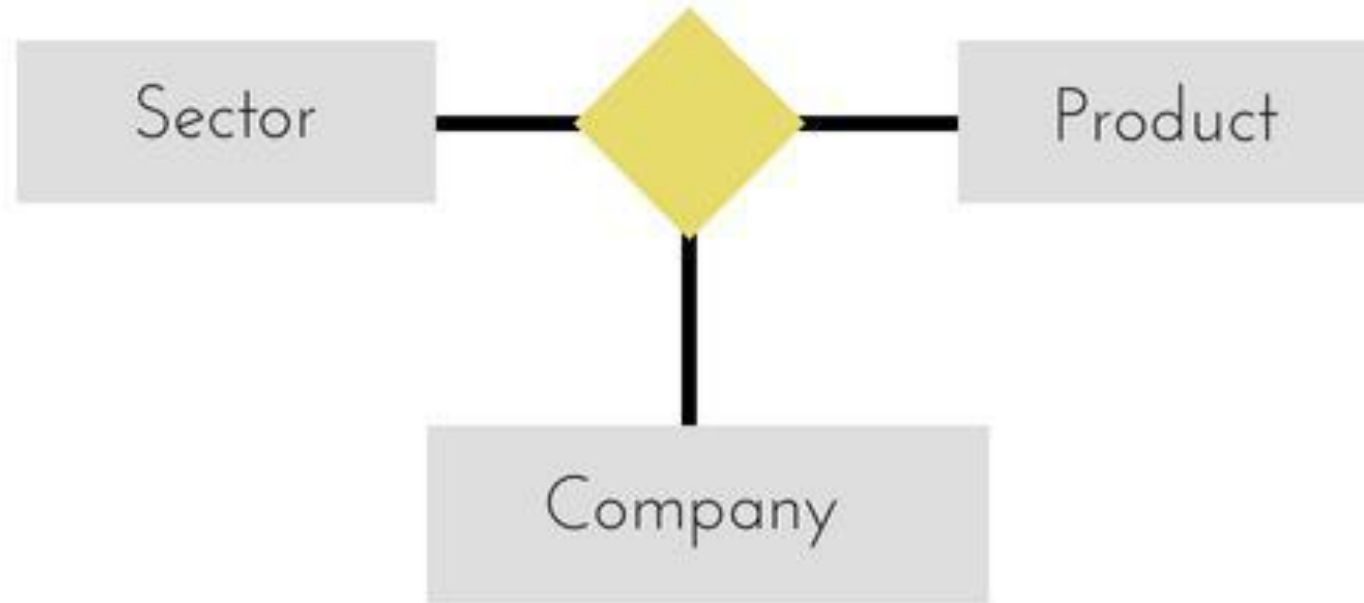
Binary Relationship



Recursive Relationship



Ternary Relationship



- The above relationship involves 3 entities.
- Company operates in Sector, producing some Products.



ER Diagram: Entity

